QuanSkill K10 · Chapter 6 Homework — Deep Learning Foundations (Pre-Gradient)

Student: Nguyen Tien Viet Hung · **Student ID:** _____ · **Date:** _____

Scope: Sections 6.1–6.14 (printed pp. 536–558). No derivatives, gradients, chain rule, `backward()`, autograd or optimizer updates are used anywhere in this notebook.

Method for every question: (1) re-read the section listed → (2) **solve by hand** (the hand solution in each question was written *before* the check cell was run) → (3) run the **instructor-provided check cell, unchanged** → (4) a small *hand vs code* cell compares my numbers with the code → (5) I explain any mismatch.

The only additions to the instructor code are the comparison cells and one parameter-count cell in Q8. Each is clearly labelled, and none uses a gradient-related API.

```
        import sys, numpy as np
        print("Python", sys.version.split()[0], "| NumPy", np.__version__)

        def compare(rows, tol=1e-3):
            # rows: list of (label, hand_value, code_value). Prints both and whether they agree within
tol.
            print(f"{'quantity':<22}{'by hand':>14}{'by code':>22}{'|diff|':>12}  match")
            for label, hand, got in rows:
                got = float(np.asarray(got).ravel()[0]) if np.size(got) == 1 else got
                diff = abs(hand - got)
                print(f"{label:<22}{hand:>14.6f}{repr(got):>22}{diff:>12.2e}  {'OK' if diff <= tol else
'MISMATCH'}")
```

```
Python 3.13.16 | NumPy 2.5.3
```

Q1. Why Deep Learning? Performance vs Complexity (10 marks)

Study first: §6.1–6.3, printed pp. 536–540.

Hand solution

Baseline: $M_{baseline} = 0.760,\ C_{baseline} = 1.00$.

1. Model A ($F1 = 0.785,\ C = 1.20$)

$$\Delta M_A = 0.785 - 0.760 = 0.025 \qquad \Delta C_A = 1.20 - 1.00 = 0.20$$

2. Model B ($F1 = 0.825,\ C = 2.10$)

$$\Delta M_B = 0.825 - 0.760 = 0.065 \qquad \Delta C_B = 2.10 - 1.00 = 1.10$$

3. Relative F1 improvement $= \dfrac{\Delta M}{M_{baseline}} \times 100\%$

$$A\text{:}\ \frac{0.025}{0.760} \times 100 = 3.2895\% \approx 3.29\% \qquad B\text{:}\ \frac{0.065}{0.760} \times 100 = 8.5526\% \approx 8.55\%$$

Extra check (my own, not asked): gain per unit of added cost. $A\text{:}\ 0.025/0.20 = 0.125$; $B\text{:}\ 0.065/1.10 \approx 0.059$. Model A buys about twice as much F1 per unit of complexity.

Instructor check cell (run unchanged):

```
# ---- instructor-provided check cell (unchanged) ----
baseline_f1 = 0.760
baseline_cost = 1.00

models = {
  "A": {"f1": 0.785, "cost": 1.20},
  "B": {"f1": 0.825, "cost": 2.10},
}

for name, m in models.items():
  delta_m = m["f1"] - baseline_f1
  delta_c = m["cost"] - baseline_cost
  relative = delta_m / baseline_f1 * 100
  print(name, "ΔM=", round(delta_m, 4),
     "ΔC=", round(delta_c, 4),
     "relative F1 gain=", round(relative, 2), "%")
```

```
A ΔM= 0.025 ΔC= 0.2 relative F1 gain= 3.29 %
B ΔM= 0.065 ΔC= 1.1 relative F1 gain= 8.55 %
```

My hand-vs-code comparison:

```
# ---- my comparison: hand values vs code (raw, unrounded floats) ----
compare([
    ("A ΔM", 0.025, models["A"]["f1"] - baseline_f1),
    ("A ΔC", 0.20, models["A"]["cost"] - baseline_cost),
    ("A relative %", 3.2895, (models["A"]["f1"] - baseline_f1) / baseline_f1 * 100),
    ("B ΔM", 0.065, models["B"]["f1"] - baseline_f1),
    ("B ΔC", 1.10, models["B"]["cost"] - baseline_cost),
    ("B relative %", 8.5526, (models["B"]["f1"] - baseline_f1) / baseline_f1 * 100),
])
```

```
quantity                    by hand              by code      |diff|  match
A ΔM                       0.025000  0.025000000000000022    2.08e-17  OK
A ΔC                       0.200000   0.19999999999999996    5.55e-17  OK
A relative %               3.289500    3.2894736842105288    2.63e-05  OK
B ΔM                       0.065000   0.06499999999999995    5.55e-17  OK
B ΔC                       1.100000                   1.1    0.00e+00  OK
B relative %               8.552600     8.552631578947361    3.16e-05  OK
```

Mismatch check. All values agree. The raw floats print as, for example, `0.19999999999999996` instead of `0.2`, because decimals such as 1.2 and 0.76 have no exact binary representation. The instructor cell hides this with `round(…, 4)`. This is a representation artefact, not a calculation error.

4. Which model to investigate first? I would investigate **Model A first**. It improves F1 by 0.025 (3.29% relative) while adding only 0.20 to the cost index, so it gives about 0.125 F1 per unit of extra complexity. Model B gives the larger gain (0.065, 8.55%), but it more than doubles the engineering cost (+1.10), and its gain per unit of cost is half of A's. Before trusting either model I would confirm on the same split that the gain is not noise. A 0.025 difference can come from a handful of examples. I would move to B only if the business value of the extra F1 clearly covers the added training, latency, monitoring and maintenance cost (§6.1: complexity must earn its place).

Q2. Artificial Neuron: Weighted Sum + Activation (15 marks)

Study first: §6.4 and ReLU/Sigmoid in §6.6, printed pp. 541–546.

$x = [0.6, -0.2, 0.9, 0.4]^T,\ w = [1.1, -0.5, 0.8, -0.3]^T,\ b = -0.25,\ z = w^T x + b$

Hand solution

5. Expand the dot product

$$
\begin{aligned}
w_1 x_1 &= 1.1 \times 0.6 = 0.66 \\
w_2 x_2 &= (-0.5) \times (-0.2) = +0.10 \\
w_3 x_3 &= 0.8 \times 0.9 = 0.72 \\
w_4 x_4 &= (-0.3) \times 0.4 = -0.12 \\
w^T x &= 0.66 + 0.10 + 0.72 - 0.12 = 1.36 \\
z &= 1.36 + (-0.25) = 1.11
\end{aligned}
$$

6. $\mathrm{ReLU}(z) = \max(0, 1.11) = 1.11$, because $z > 0$ and the value passes through unchanged.

7. Sigmoid $\sigma(1.11) = \dfrac{1}{1+e^{-1.11}}$

$$
e^{-1.11} = e^{-1} \cdot e^{-0.11} \approx 0.367879 \times 0.895834 \approx 0.329559 \;\Rightarrow\; \sigma(1.11) = \frac{1}{1.329559} \approx 0.7521
$$

8. The **weights** say how strongly, and in which direction (positive or negative), each input feature pushes the neuron's output. The **bias** shifts the weighted sum up or down, so the neuron can activate (or stay quiet) even when the weighted inputs alone would not reach that point.

9. No. I only *evaluated* the neuron with fixed, given parameters. Learning means changing w and b to reduce a loss over many examples. That needs a loss, gradients and an update step, and none of these happened here. After this calculation the parameters are exactly what they were before.

Instructor check cell (run unchanged):

```
        # ---- instructor-provided check cell (unchanged) ----
        import numpy as np

        x = np.array([0.6, -0.2, 0.9, 0.4])
        w = np.array([1.1, -0.5, 0.8, -0.3])
        b = -0.25

        z=w@x+b
        relu = max(0.0, z)
        sigmoid = 1 / (1 + np.exp(-z))

        print("z =", z)
        print("ReLU(z) =", relu)
        print("sigmoid(z) =", sigmoid)
```

```
z = 1.1099999999999999
ReLU(z) = 1.1099999999999999
sigmoid(z) = 0.7521291114395701
```

My hand-vs-code comparison:

```
        # ---- my comparison ----
        print("element-wise products w*x =", w * x)
        compare([("z", 1.11, z), ("ReLU(z)", 1.11, relu), ("sigmoid(z)", 0.752130, sigmoid)], tol=1e-5)
```

```
element-wise products w*x = [ 0.66  0.1   0.72 -0.12]
quantity                     by hand              by code      |diff|  match
z                           1.110000    1.1099999999999999    2.22e-16  OK
ReLU(z)                     1.110000    1.1099999999999999    2.22e-16  OK
sigmoid(z)                  0.752130    0.7521291114395701    8.89e-07  OK
```

Mismatch check. z prints as `1.1099999999999999` instead of exactly 1.11. This is floating-point rounding in the products and sums (for example, 0.66 and 0.72 cannot be stored exactly in binary); the difference is about 10^{-16}. My hand sigmoid was 0.752130, while the code gives 0.7521291..., so they differ in the 6th decimal. The cause is that I rounded $e^{-0.11}$ and e^{-1} to 6 digits by hand. To the 4 decimal places required, both give **0.7521**.

Q3. Why Nonlinearity? + Activation Functions (15 marks)

Study first: §6.5–6.6, printed pp. 542–546.

Part A — Algebraic proof (hand)

10. Start from the two linear layers

$$h = W_1 x + b_1, \qquad y = W_2 h + b_2.$$

Substitute h into the second equation and distribute W_2 (matrix multiplication distributes over addition):

$$y = W_2 (W_1 x + b_1) + b_2 = (W_2 W_1) x + (W_2 b_1 + b_2) .$$

Define

$$W' = W_2 W_1, \qquad b' = W_2 b_1 + b_2 \quad \Rightarrow \quad y = W' x + b'.$$

Shapes: if $x \in \mathbb{R}^d$, $W_1 \in \mathbb{R}^{h \times d}$ and $W_2 \in \mathbb{R}^{k \times h}$, then $W' \in \mathbb{R}^{k \times d}$ and $b' \in \mathbb{R}^k$, which is exactly the form of **one** linear layer.

11. However many purely linear layers I stack, the product of their matrices is still a single matrix and the bias terms still add up to a single vector, so the whole stack is equivalent to one linear map. It can only draw straight (hyper-plane) boundaries. The extra layers add parameters and cost but no new *kind* of function. A nonlinear activation between the layers is what prevents this collapse and lets the network bend its decision boundary.

Part B — Activation behaviour (hand)

12. Using $\sigma (z) = 1/ (1 + e^{-z})$, $\tanh (z) = (e^{z} - e^{-z}) / (e^{z} + e^{-z})$ and $\mathrm{ReLU} (z) = \max (0, z)$:

z	Sigmoid	Tanh	ReLU
−3	$1/ (1 + e^{3}) = 1/21.0855 = 0.0474$	−0.9951	0
−1	$1/ (1 + e^{1}) = 1/3.7183 = 0.2689$	−0.7616	0
0	$1/ (1 + 1) = 0.5000$	0.0000	0
1	$1/ (1 + e^{-1}) = 1/1.3679 = 0.7311$	0.7616	1
3	$1/ (1 + e^{-3}) = 1/1.0498 = 0.9526$	0.9951	3

Hand detail for tanh(1): $(2.7183 - 0.3679) / (2.7183 + 0.3679) = 2.3504/3.0862 = 0.7616$. tanh is odd, so tanh(−1) = −0.7616. Hand detail for tanh(3): $(20.0855 - 0.0498) / (20.0855 + 0.0498) = 20.0357/20.1353 = 0.9951$.

13. The chapter's practical choice for the first MLP hidden layers is **ReLU**. It is simple, cheap and does not saturate for positive inputs. **Caution:** a ReLU unit outputs 0 for every negative input. If its pre-activation stays negative for all examples, the unit becomes **"dead"**: it always outputs 0 and contributes nothing. (Sigmoid and tanh have the opposite caution: they **saturate** near their limits, as the ±3 rows above show.)

Instructor check cell (run unchanged):

```
# ---- instructor-provided check cell (unchanged) ----
import numpy as np

z = np.array([-3., -1., 0., 1., 3.])

sigmoid = 1 / (1 + np.exp(-z))
tanh = np.tanh(z)
relu = np.maximum(0, z)

for zi, si, ti, ri in zip(z, sigmoid, tanh, relu):
  print(f"z={zi:>4.1f} | sigmoid={si:.4f} | tanh={ti:.4f} | ReLU={ri:.4f}")
```

```
z=-3.0 | sigmoid=0.0474 | tanh=-0.9951 | ReLU=0.0000
z=-1.0 | sigmoid=0.2689 | tanh=-0.7616 | ReLU=0.0000
z= 0.0 | sigmoid=0.5000 | tanh=0.0000 | ReLU=0.0000
z= 1.0 | sigmoid=0.7311 | tanh=0.7616 | ReLU=1.0000
z= 3.0 | sigmoid=0.9526 | tanh=0.9951 | ReLU=3.0000
```

My hand-vs-code comparison:

```
        # ---- my comparison (hand table vs code) ----
        hand_sig = [0.0474, 0.2689, 0.5, 0.7311, 0.9526]
        hand_tanh = [-0.9951, -0.7616, 0.0, 0.7616, 0.9951]
        hand_relu = [0, 0, 0, 1, 3]
        print("max |sigmoid diff| =", np.max(np.abs(sigmoid - hand_sig)).round(6))
        print("max |tanh diff|    =", np.max(np.abs(tanh - hand_tanh)).round(6))
        print("max |ReLU diff|    =", np.max(np.abs(relu - hand_relu)))

        # my own numerical illustration of Q10: two linear layers == one linear layer
        rng = np.random.default_rng(0)
        W1, b1 = rng.normal(size=(3, 4)), rng.normal(size=3)
        W2, b2 = rng.normal(size=(2, 3)), rng.normal(size=2)
        xv = rng.normal(size=4)
        two_layers = W2 @ (W1 @ xv + b1) + b2
        one_layer = (W2 @ W1) @ xv + (W2 @ b1 + b2)
        print("two linear layers :", two_layers)
        print("W'x + b'          :", one_layer, "| equal:", np.allclose(two_layers, one_layer))
```

```
max |sigmoid diff| = 4.1e-05
max |tanh diff|    = 4.5e-05
max |ReLU diff|    = 0.0
two linear layers : [ 4.35953641 -2.00023577]
W'x + b'          : [ 4.35953641 -2.00023577] | equal: True
```

Mismatch check. The hand table matches the code to 4 decimals; the largest difference is below 5×10^{-5} and comes only from rounding to 4 decimals. The extra numerical check confirms Q10: for random matrices, two stacked linear layers give exactly the same output as the single layer $W'x + b'$.

Q4. From One Neuron to a Layer + Hidden Representation (15 marks)

Study first: §6.7–6.9, printed pp. 546–550.

$x = [1.0, 0.5, -1.0, 2.0]^T$, 3 hidden units, $z = Wx + b$, $h = \mathrm{ReLU}(z)$.

Hand solution

14. Shapes

object	math shape	NumPy shape
x	4×1 (in $\mathbb{R}^4$)	`(4,)`
W	3×4 (one row per hidden unit, one column per input)	`(3, 4)`
b	3×1	`(3,)`
z	3×1	`(3,)`
h	3×1	`(3,)`

Rule: $(3 \times 4)(4 \times 1) = 3 \times 1$, then $+b$ (3×1) and ReLU keep the shape.

15. Row 1 in full detail (unit 1, $w_1 = [0.5, 1.0, -0.5, 0.2]$, $b_1 = 0.1$)

$$\begin{aligned} z_1 &= 0.5(1.0) + 1.0(0.5) + (-0.5)(-1.0) + 0.2(2.0) + 0.1 \\ &= 0.5 + 0.5 + 0.5 + 0.4 + 0.1 = 2.0 \end{aligned}$$

Row 2: $z_2 = -0.3(1.0) + 0.8(0.5) + 0.4(-1.0) + (-0.6)(2.0) - 0.2 = -0.3 + 0.4 - 0.4 - 1.2 - 0.2 = -1.7$

Row 3: $z_3 = 1.2(1.0) + (-0.4)(0.5) + 0.1(-1.0) + 0.5(2.0) + 0.3 = 1.2 - 0.2 - 0.1 + 1.0 + 0.3 = 2.2$

$$z = [2.0, -1.7, 2.2]^T$$

16. $h = \mathrm{ReLU}\,(\,z\,) \;=\; [\;\max\;(\,0, 2.0\,)\,,\; \max\;(\,0, -1.7\,)\,,\; \max\;(\,0, 2.2\,)\;]^{T} = [\,2.0,\; \mathbf{0},\; 2.2\,]^{\mathbf{T}}$. Unit 2 is switched off for this input.

17. h is not a copy of x. It lives in a different space (3 numbers instead of 4), and each entry is a new feature: a weighted combination of *all* inputs, passed through a nonlinearity. For example, h_1 responds to a specific pattern of the four inputs, and unit 2 stays silent (0) for this input. The network computes h internally and never shows it as input or output, which is why it is called *hidden*. Once the weights are trained, these units become learned features chosen to make the final prediction easier. This replaces the hand-engineered features of Chapter 5.

Instructor check cell (run unchanged):

```
        # ---- instructor-provided check cell (unchanged) ----
        import numpy as np

        x = np.array([1.0, 0.5, -1.0, 2.0])
        W = np.array([
           [ 0.5, 1.0, -0.5, 0.2],
           [-0.3, 0.8, 0.4, -0.6],
           [ 1.2, -0.4, 0.1, 0.5],
        ])
        b = np.array([0.1, -0.2, 0.3])

        z=W@x+b
        h = np.maximum(0, z)

        print("x shape:", x.shape)
        print("W shape:", W.shape)
        print("b shape:", b.shape)
        print("z =", z)
        print("h = ReLU(z) =", h)
```

```
x shape: (4,)
W shape: (3, 4)
b shape: (3,)
z = [ 2.  -1.7  2.2]
h = ReLU(z) = [2.  0.  2.2]
```

My hand-vs-code comparison:

```
        # ---- my comparison ----
        print("row-1 products:", W[0] * x, "sum =", (W[0] * x).sum(), "+ b1 =", (W[0] * x).sum() + b[0])
        print("z exact repr  :", [repr(v) for v in z])
        compare([("z1", 2.0, z[0]), ("z2", -1.7, z[1]), ("z3", 2.2, z[2]),
                 ("h1", 2.0, h[0]), ("h2", 0.0, h[1]), ("h3", 2.2, h[2])])
```

```
row-1 products: [0.5 0.5 0.5 0.4] sum = 1.9 + b1 = 2.0
z exact repr  : ['np.float64(2.0)', 'np.float64(-1.7)', 'np.float64(2.1999999999999997)']
quantity                     by hand               by code      |diff|  match
z1                          2.000000                   2.0    0.00e+00  OK
z2                         -1.700000                  -1.7    0.00e+00  OK
z3                          2.200000    2.1999999999999997    4.44e-16  OK
h1                          2.000000                   2.0    0.00e+00  OK
h2                          0.000000                   0.0    0.00e+00  OK
h3                          2.200000    2.1999999999999997    4.44e-16  OK
```

Mismatch check. No real mismatch. `repr` shows the usual tiny floating-point tails (for example z_3 = `2.1999999999999997`), because values such as 0.3, 0.4 and 0.6 are not exact in binary. NumPy prints vectors as 1-D arrays `(4,)` and `(3,)`, whereas on paper I wrote column vectors 4×1 and 3×1. The numbers are the same; only the notation differs.

Q5. Depth and Parameter Counting (15 marks)

Study first: §6.10–6.11, printed pp. 550–553.

Dense layer: $N = d \times h + h = h\,(\,d + 1\,)$ (weights + one bias per output unit).

Hand solution

18. Network A: $6 \to 16 \to 8 \to 1$

$$\begin{aligned} L_1 &: 6 \times 16 + 16 = 96 + 16 = 112 \\ L_2 &: 16 \times 8 + 8 = 128 + 8 = 136 \\ L_3 &: 8 \times 1 + 1 = 8 + 1 = 9 \\ \text{Total}_A &= 112 + 136 + 9 = \mathbf{257} \end{aligned}$$

19. Network B: $6 \to 64 \to 32 \to 1$

$$\begin{aligned} L_1 &: 6 \times 64 + 64 = 384 + 64 = 448 \\ L_2 &: 64 \times 32 + 32 = 2048 + 32 = 2080 \\ L_3 &: 32 \times 1 + 1 = 33 \\ \text{Total}_B &= 448 + 2080 + 33 = \mathbf{2561} \end{aligned}$$

20. $\frac{2561}{257} = 9.965 \approx$ **9.97 times** more trainable parameters (almost 10×). Most of the growth is in the 64×32 middle layer: parameters grow with the *product* of adjacent widths.

21. Three engineering risks or costs that grow with depth, width and capacity 1. **Overfitting / memorisation:** more parameters can fit noise and label errors in a small dataset, so the model needs more data, regularisation and a strictly protected validation/test split. 2. **Compute, memory and latency:** more weights mean more multiply-adds per forward pass, more memory, slower inference and higher serving cost. That is the ΔC from Q1. 3. **Harder to train and debug:** deeper stacks are more sensitive to initialisation and hyperparameters, have more ways to fail silently (for example dead units), and are harder to interpret and maintain. This is exactly why the chapter insists that complexity must earn its place against the classical baseline.

Instructor check cell (run unchanged):

```
        # ---- instructor-provided check cell (unchanged) ----
        def dense_params(in_dim, out_dim):
          return in_dim * out_dim + out_dim

        def mlp_params(sizes):
          per_layer = []
          for a, b in zip(sizes[:-1], sizes[1:]):
            per_layer.append(dense_params(a, b))
          return per_layer, sum(per_layer)

        for name, sizes in {
           "Network A": [6, 16, 8, 1],
           "Network B": [6, 64, 32, 1],
        }.items():
           layer_counts, total = mlp_params(sizes)
           print(name, sizes)
           print("per layer =", layer_counts)
           print("total =", total)
           print()
```

```
Network A [6, 16, 8, 1]
per layer = [112, 136, 9]
total = 257

Network B [6, 64, 32, 1]
per layer = [448, 2080, 33]
total = 2561
```

My hand-vs-code comparison:

```
# ---- my comparison ----
(_, tA), (_, tB) = mlp_params([6, 16, 8, 1]), mlp_params([6, 64, 32, 1])
assert mlp_params([6, 16, 8, 1])[0] == [112, 136, 9] and tA == 257
assert mlp_params([6, 64, 32, 1])[0] == [448, 2080, 33] and tB == 2561
print("hand totals match: A =", tA, ", B =", tB, ", ratio B/A =", round(tB / tA, 3))
```

```
hand totals match: A = 257 , B = 2561 , ratio B/A = 9.965
```

Mismatch check. Parameter counts are integers, so hand and code agree exactly (257, 2561, ratio 9.965).

Q6. Full Forward Pass Through a Tiny MLP (15 marks)

Study first: §6.8 and §6.12, printed p. 548 and pp. 553–554.

$x = [1.0, -2.0, 0.5]^T$. Architecture $3 \to 3 \to 2 \to 1$ with ReLU, ReLU, then sigmoid on the output logit.

Hand solution

22. Layer 1 $z^{(1)} = W^{(1)} x + b^{(1)}$

$$
\begin{aligned}
z_1^{(1)} &= 0.5\,(1) + (-0.2)\,(-2) + 0.4\,(0.5) + 0.1 = 0.5 + 0.4 + 0.2 + 0.1 = 1.2 \\
z_2^{(1)} &= -0.3\,(1) + 0.8\,(-2) + (-0.5)\,(0.5) - 0.1 = -0.3 - 1.6 - 0.25 - 0.1 = -2.25 \\
z_3^{(1)} &= 0.7\,(1) + 0.1\,(-2) + 0.2\,(0.5) + 0.0 = 0.7 - 0.2 + 0.1 = 0.6
\end{aligned}
$$

$$
z^{(1)} = [1.2,\ -2.25,\ 0.6]^T, \qquad h^{(1)} = \mathrm{ReLU}\left(z^{(1)}\right) = [1.2,\ \mathbf{0},\ 0.6]^{\mathbf{T}}
$$

23. Layer 2 $z^{(2)} = W^{(2)} h^{(1)} + b^{(2)}$

$$
\begin{aligned}
z_1^{(2)} &= 0.6\,(1.2) + (-0.4)\,(0) + 0.3\,(0.6) + 0.05 = 0.72 + 0 + 0.18 + 0.05 = 0.95 \\
z_2^{(2)} &= -0.2\,(1.2) + 0.5\,(0) + 0.7\,(0.6) - 0.15 = -0.24 + 0 + 0.42 - 0.15 = 0.03
\end{aligned}
$$

$$
z^{(2)} = [0.95,\ 0.03]^T, \qquad h^{(2)} = \mathrm{ReLU}\left(z^{(2)}\right) = [0.95,\ 0.03]^{\mathbf{T}} \text{ (both positive, unchanged)}
$$

24. Output logit

$$
z^{(3)} = 0.8\,(0.95) + (-0.6)\,(0.03) + 0.2 = 0.76 - 0.018 + 0.2 = 0.942
$$

25. Probability

$$
e^{-0.942} = e^{-1} \cdot e^{0.058} \approx 0.367879 \times 1.059715 \approx 0.389847, \qquad p = \frac{1}{1 + 0.389847} = \frac{1}{1.389847} \approx 0.7195
$$

Has the network learned anything? No. This is one forward pass with fixed, hand-given weights. $p \approx 0.72$ is just what these particular numbers produce. No label was compared, no loss was computed and no weight changed, so the 0.72 is not a trained prediction and should not be trusted as a probability of anything.

Instructor check cell (run unchanged):

```
        # ---- instructor-provided check cell (unchanged) ----
        import numpy as np

        x = np.array([1.0, -2.0, 0.5])
        W1 = np.array([
           [ 0.5, -0.2, 0.4],
           [-0.3, 0.8, -0.5],
           [ 0.7, 0.1, 0.2],
        ])
        b1 = np.array([0.1, -0.1, 0.0])

        W2 = np.array([
           [ 0.6, -0.4, 0.3],
           [-0.2, 0.5, 0.7],
        ])
        b2 = np.array([0.05, -0.15])

        W3 = np.array([[0.8, -0.6]])
        b3 = np.array([0.2])

        relu = lambda v: np.maximum(0, v)

        z1 = W1 @ x + b1
        h1 = relu(z1)
        z2 = W2 @ h1 + b2
        h2 = relu(z2)
        z3 = W3 @ h2 + b3
        p = 1 / (1 + np.exp(-z3))

        print("z1 =", z1)
        print("h1 =", h1)
        print("z2 =", z2)
        print("h2 =", h2)
        print("raw logit z3 =", z3)
        print("sigmoid probability p =", p)
```

```
z1 = [ 1.2  -2.25  0.6 ]
h1 = [1.2 0.  0.6]
z2 = [0.95 0.03]
h2 = [0.95 0.03]
raw logit z3 = [0.942]
sigmoid probability p = [0.71950347]
```

My hand-vs-code comparison:

```
        # ---- my comparison ----
        compare([("z1[0]", 1.2, z1[0]), ("z1[1]", -2.25, z1[1]), ("z1[2]", 0.6, z1[2]),
                 ("h1[1]", 0.0, h1[1]), ("z2[0]", 0.95, z2[0]), ("z2[1]", 0.03, z2[1]),
                 ("z3", 0.942, z3), ("p", 0.719504, p)], tol=1e-5)
        print("shapes:", z1.shape, z2.shape, z3.shape, "| z3 is a length-1 array, so it prints as", z3)
```

```
quantity                    by hand              by code      |diff|  match
z1[0]                      1.200000    1.2000000000000002    2.22e-16  OK
z1[1]                     -2.250000   -2.2500000000000004    4.44e-16  OK
z1[2]                      0.600000                   0.6    0.00e+00  OK
h1[1]                      0.000000                   0.0    0.00e+00  OK
z2[0]                      0.950000    0.9500000000000001    1.11e-16  OK
z2[1]                      0.030000  0.029999999999999916    8.33e-17  OK
z3                         0.942000    0.9420000000000002    2.22e-16  OK
p                          0.719504     0.719503471011179    5.29e-07  OK
shapes: (3,) (2,) (1,) | z3 is a length-1 array, so it prints as [0.942]
```

Mismatch check. All values agree to at least 5 decimals. Small differences: (1) the code prints values such as `0.95` or `0.03` with floating-point tails in `repr` (for example $z_2^{(2)}$ = `0.029999999999999916`); (2) my hand $p = 0.719504$ vs code $0.7195034\ldots$, which differ in the 7th decimal because I rounded e^{-1} and $e^{0.058}$ by hand; (3) the code prints $z^{(3)}$ and p as `[0.942]`, a length-1 array, because $W^{(3)}$ has shape `(1, 2)`. On paper I wrote a scalar. The output layer has one unit, so the result is a 1×1 vector. Same value, different container.

Q7. From One Example to a Batch (10 marks)

Study first: §6.13, printed pp. 554–556.

$Z = XW^T + \mathbf{1}b^T$, with $B = 4,\ d = 3,\ h = 2$.

Hand solution

26. Shapes

object	shape	meaning
X	4×3 ($B \times d$)	4 examples (rows) × 3 features
W	2×3 ($h \times d$)	2 units, each with 3 weights
W^T	3×2 ($d \times h$)	transposed so that XW^T is defined
b	$(2,\ )$, i.e. h	one bias per unit; $\mathbf{1}b^T$ copies it to all 4 rows
Z	4×2 ($B \times h$)	$(4 \times 3)\ (3 \times 2) = 4 \times 2$

27. First row $Z[0]$, using $x_0 = [1, 0, 2]$:

$$\begin{aligned} Z[0,0] &= 1(0.5) + 0(-0.2) + 2(0.3) + 0.1 = 0.5 + 0 + 0.6 + 0.1 = 1.2 \\ Z[0,1] &= 1(-0.4) + 0(0.7) + 2(0.1) - 0.2 = -0.4 + 0 + 0.2 - 0.2 = -0.4 \end{aligned} \quad \Rightarrow \quad Z[0] = [1.2,\ -0.4]$$

I also did the other rows by hand before running the checker:
$Z[1] = [-0.2 - 0.3 + 0.1,\ 0.7 - 0.1 - 0.2] = [-0.4,\ 0.4]$; $Z[2] = [1.0 - 0.2 + 0.1,\ -0.8 + 0.7 - 0.2] = [0.9,\ -0.3]$; $Z[3] = [-0.5 - 0.4 + 0.3 + 0.1,\ 0.4 + 1.4 + 0.1 - 0.2] = [-0.5,\ 1.7]$.

28. (see the code output below) **Each row of Z holds the two hidden units' pre-activation values for one example in the batch.** Row i is exactly what the single-example formula $Wx_i + b$ gives for example i, computed for all 4 examples at once in one matrix multiplication.

Instructor check cell (run unchanged):

```python
# ---- instructor-provided check cell (unchanged) ----
import numpy as np

X = np.array([
   [ 1., 0., 2.],
   [ 0., 1., -1.],
   [ 2., 1., 0.],
   [-1., 2., 1.],
])
W = np.array([
   [ 0.5, -0.2, 0.3],
   [-0.4, 0.7, 0.1],
])
b = np.array([0.1, -0.2])

Z = X @ W.T + b

print("X shape =", X.shape)
print("W shape =", W.shape)
print("W.T shape =", W.T.shape)
print("b shape =", b.shape)
print("Z shape =", Z.shape)
print("Z =\n", Z)
```

```
X shape = (4, 3)
W shape = (2, 3)
W.T shape = (3, 2)
b shape = (2,)
Z shape = (4, 2)
Z =
 [[ 1.2 -0.4]
 [-0.4  0.4]
 [ 0.9 -0.3]
 [-0.5  1.7]]
```

My hand-vs-code comparison:

```python
# ---- my comparison ----
Z_hand = np.array([[1.2, -0.4], [-0.4, 0.4], [0.9, -0.3], [-0.5, 1.7]])
print("max |Z - Z_hand| =", np.max(np.abs(Z - Z_hand)))
# batch formula == looping the single-example formula row by row
Z_loop = np.stack([W @ X[i] + b for i in range(len(X))])
print("batch result equals per-example loop:", np.allclose(Z, Z_loop))
```

```
max |Z - Z_hand| = 2.220446049250313e-16
batch result equals per-example loop: True
```

Mismatch check. All 8 entries match my hand values, apart from floating-point tails smaller than 10^{-15}. In the code, `+ b` is NumPy **broadcasting**: the `(2,)` bias is added to each of the 4 rows, which is exactly the $\mathbf{1}b^T$ term in the book's formula. The loop check confirms that the batch formula is just the single-example formula applied to every row.

Q8. PyTorch Mental Model: Tensor → Module → Forward Pass (5 marks)

Study first: §6.14, printed pp. 556–558.

Prediction by hand (before running)

`nn.Linear(in, out)` stores `weight` with shape **(out, in)**, the same convention as $W \in \mathbb{R}^{h \times d}$, and `bias` with shape **(out,)**. So I expect: input `(4, 3)`; weights `(32, 3)`, `(8, 32)`, `(1, 8)`; biases `(32,)`, `(8,)`, `(1,)`; output `(4, 1)`, one logit per example. Parameter count: $3 \cdot 32 + 32 = 128$, $32 \cdot 8 + 8 = 264$, $8 \cdot 1 + 1 = 9$, total **401**.

Instructor check cell (run unchanged):

```
        # ---- instructor-provided check cell (unchanged) ----
        import torch
        import torch.nn as nn

        x = torch.tensor([
           [0.8, 0.5, 0.2],
           [0.1, 0.4, 0.9],
           [0.6, 0.3, 0.7],
           [0.2, 0.9, 0.1],
        ], dtype=torch.float32)

        class SmallMLP(nn.Module):
          def __init__(self, input_dim):
            super().__init__()
            self.network = nn.Sequential(
               nn.Linear(input_dim, 32),
               nn.ReLU(),
               nn.Linear(32, 8),
               nn.ReLU(),
               nn.Linear(8, 1),
            )

          def forward(self, x):
            return self.network(x)

        model = SmallMLP(input_dim=3)
        logits = model(x)

        print("input shape:", x.shape)
        print("output/logit shape:", logits.shape)
        print("\nTrainable parameters:")
        for name, parameter in model.named_parameters():
          print(name, parameter.shape, "requires_grad=", parameter.requires_grad)
```

```
input shape: torch.Size([4, 3])
output/logit shape: torch.Size([4, 1])

Trainable parameters:
network.0.weight torch.Size([32, 3]) requires_grad= True
network.0.bias torch.Size([32]) requires_grad= True
network.2.weight torch.Size([8, 32]) requires_grad= True
network.2.bias torch.Size([8]) requires_grad= True
network.4.weight torch.Size([1, 8]) requires_grad= True
network.4.bias torch.Size([1]) requires_grad= True
```

```
        # ---- my addition: parameter count and the raw logits (forward only; no loss, no backward, no
optimizer) ----
        total = sum(p.numel() for p in model.parameters())
        print("PyTorch", torch.__version__, "| total parameters =", total, "(hand prediction 401)")
        print("raw logits for the 4 examples:", [round(v, 4) for v in logits.flatten().tolist()])
        print("Note: these logit values come from RANDOM initial weights and change on every run; the
shapes do not.")
```

```
PyTorch 2.14.1+cu130 | total parameters = 401 (hand prediction 401)
raw logits for the 4 examples: [0.1101, 0.0622, 0.1015, 0.0744]
Note: these logit values come from RANDOM initial weights and change on every run; the shapes do not.
```

29. Input shape `torch.Size([4, 3])` (4 examples × 3 features); output/logit shape `torch.Size([4, 1])` (one raw logit per example).

30. Layer shapes printed: `network.0.weight (32, 3)`, `network.0.bias (32)`; `network.2.weight (8, 32)`, `network.2.bias (8)`; `network.4.weight (1, 8)`, `network.4.bias (1)`. Indices 1 and 3 are the ReLU modules, which have no parameters, so they are not listed. Total = **401** parameters, matching my hand count.

31. Calling `model(x)` only runs the forward pass: it pushes the batch through Linear → ReLU → Linear → ReLU → Linear with whatever weights the layers were randomly initialised with. Training would need three more things that are deliberately absent: a **loss** that compares the logits with true labels, a way to work out how each weight should change (next class), and an **optimizer step** that actually updates the weights. `requires_grad=True` only means the parameters *can* be trained later; nothing has been updated, so the model knows nothing yet and its logits are arbitrary.

Mismatch check. All shapes match my prediction. The one difference from the book's notation: PyTorch stores the weight as `(out, in)` and computes $xW^T + b$ for a batch, which is exactly the Q7 formula $Z = XW^T + \mathbf{1}b^T$. The printed logit values are not reproducible because there is no seed, but the homework only asks for shapes.

Final submission checklist

- [x] I solved all mathematical parts before using the Python checker (hand solution above each check cell).
- [x] I included intermediate calculations, not only final numbers.
- [x] I included the code output for every check section (outputs saved in this notebook).
- [x] I did not use gradients, `backward()`, autograd or optimizer updates.
- [x] I answered the interpretation questions in my own words.
- [x] I reviewed the specified Chapter 6 section/page before each question.
- [x] This notebook runs top to bottom without errors.

Ready for next class: forward computation, shapes, hidden representations and parameter counts are all checked by hand and by code. The open question for next session is how the network finds out *which* weights to change, and by how much.